

B.Tech Project · Full Revision Notes

Bhunaga

भुनागा — a trilingual agricultural RAG assistant for Indian farmers

What it is Ask a farming question in Marathi, Hindi or English; get an answer in the same language, grounded in documents, not model memory.

Scale ~8,300 lines of Python · 24 documents · 114 chunks · 10 test files

Stack BGE-M3 · Qdrant / numpy · Groq GPT-OSS 120B · Streamlit · FastAPI

Cost Zero. One free API key, everything else free or local.

These notes cover **why** the project exists, **how** every part works, **what** each term means, and the **questions you will be asked** about it. Read §4 before §5 if the vocabulary is new.

1 Why this project exists

The information a farmer needs already exists and is already public. It just never reaches them in a form they can use.

The four-part problem

PROBLEM	WHAT IT ACTUALLY MEANS
Wrong language	Package-of-practices documents, ICAR advisories and nearly every digital assistant are English-first. A farmer who thinks in Marathi cannot query them.
Lossy translation	The standard workaround — translate the question to English, search, translate the answer back — loses domain vocabulary at every hop. Agricultural terms often have no clean English equivalent.
Script ambiguity	Hindi and Marathi share the Devanagari script. Tools that guess language from characters confuse the two, then answer in the wrong one.
Scattered sources	Price, weather, pest and agronomic data live on four unconnected portals. One real question — *"rain is coming, should I spray?"* — needs all four joined by hand.

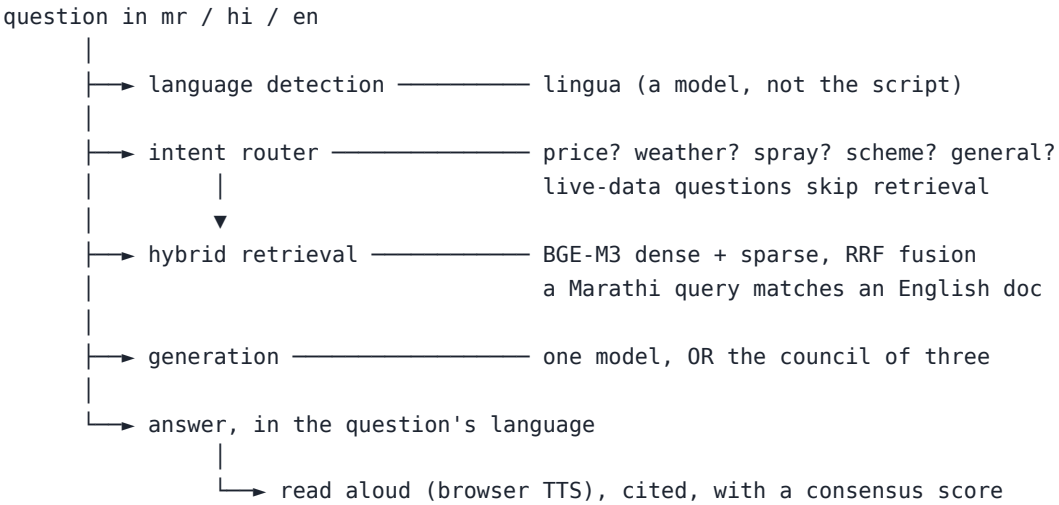
WHY NOT JUST USE CHATGPT / A PLAIN LLM?

Two reasons, and both are fatal for this use case. **It hallucinates specifics.** A wrong fertiliser dose reads exactly like a right one — fluent, confident, and costing a farmer a season. **It has no live data.** Today's mandi price and this week's forecast are simply not in the model's weights. Grounding the answer in retrieved documents fixes the first; routing live questions to real APIs fixes the second.

THE CONSEQUENCE NOBODY STATES

When no trustworthy advice is available, the nearest source of pest advice becomes the person selling the pesticide. That is a structural conflict of interest, and it is why the Government's own National Pest Surveillance System was launched — explicitly to reduce farmers' dependence on pesticide retailers.

2 What it does — one page



FIVE THINGS A FARMER CAN DO

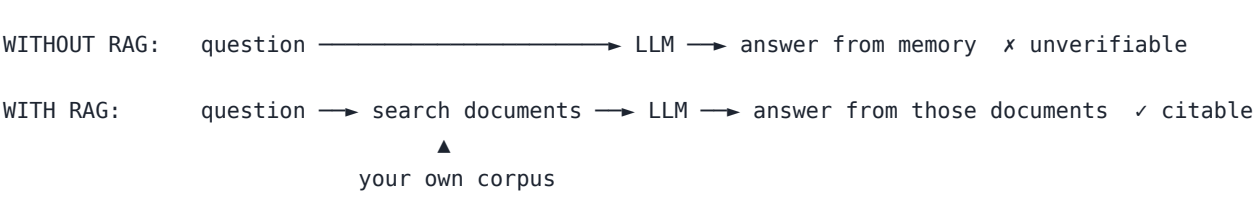
- **Ask** — by voice, by tapping a picture card, or by typing. All three are front doors, not fallbacks.
- **Today** — a daily brief: weather, spray window, soil, what to do now.
- **Money** — cost of cultivation, revenue, profit, break-even.
- **Market** — mandi prices with source and timestamp.
- **Listen** — every answer can be read aloud.

WHY THREE WAYS TO ASK

An advisory app that assumes literacy excludes most of the people it is meant to serve. Voice input *without* voice output is half a bridge: the farmer speaks the question and then lands on a screen full of text they cannot read. That is why the Listen button matters more than it looks.

3 The core idea — RAG

Retrieval-Augmented Generation. Instead of asking a language model what it remembers, you first *retrieve* relevant passages from a trusted document collection, then hand those passages to the model and ask it to answer *from them*.



Why RAG rather than fine-tuning

APPROACH	TRADE-OFF
Fine-tuning	Bakes knowledge into weights. Expensive, needs retraining to update, and the model still cannot cite where an answer came from.
RAG	**Knowledge stays in files. Add a PDF and the system knows it immediately. Every answer can name its source. Cheap. ← chosen**
Long context	Paste everything into the prompt. Works for small corpora, costs tokens linearly, and degrades as the corpus grows.

LIKELY QUESTION

"Why not fine-tune on agricultural data?" — Because advisories change every season and a fine-tuned model cannot tell you which document a dose came from. Traceability is the whole point when the answer is a chemical quantity.

4 Terminology — everything used in this project

4.1 Language models

LLM Large Language Model	A model trained to predict text. Here: GPT-OSS 120B, served by Groq. It writes the final answer; it does not decide the facts.
Token	The unit a model reads and bills in — roughly a word-piece. Devanagari costs several times more tokens than Latin for the same meaning, which is why Marathi answers hit limits first.
Context window	How much text the model can see at once — the question plus all retrieved chunks plus the answer.
Hallucination	A fluent, confident, wrong answer. The specific danger here is an invented dose or spray interval.
Grounding	Tying the answer to retrieved evidence. Measured in this project as the fraction of numbers in the answer that appear in the sources.
Temperature	Randomness of generation. Set to `0.2` — advice should be stable, not creative.
Reasoning model	A model that spends tokens thinking before it writes. GPT-OSS does this, out of the same budget as the answer — so too small a limit returns empty content, not an error.
reasoning_effort	`low / medium / high`. Set to `low`: fewer thinking tokens, lower cost, less risk of the answer being squeezed out.
TPD / RPM	Tokens-per-day and requests-per-minute rate limits. The free tier gives 200,000 TPD — a real design constraint, not a footnote.

4.2 Embeddings and vectors

Embedding	A piece of text turned into a list of numbers (a vector) such that similar meanings land near each other.
Vector / dimension	The list itself. BGE-M3 produces **1024 dimensions** per chunk. The index here is a `(114, 1024)` float32 matrix.
Cosine similarity	Angle between two vectors — the standard measure of "how close in meaning".
Shared vector space	The idea the whole project rests on. A multilingual encoder places युरिया, urea and यूरिया at nearly the same point. So a Marathi question retrieves an English document with no translation step.

BGE-M3	The embedding model used. Multilingual, and emits dense *and* sparse vectors from one pass — so no separate keyword index is needed.
Dense vector	Captures meaning. Finds "how much nitrogen" when the document says "urea dose".
Sparse vector	Captures exact words. Finds "PM-KISAN" when that exact string matters. Conceptually BM25 / TF-IDF.
Chunking	Splitting documents into retrievable pieces. 24 documents → 114 chunks here.

WHY YOU NEED BOTH DENSE AND SPARSE

Dense alone misses exact identifiers — scheme names, chemical names, numbers. Sparse alone misses paraphrase — a farmer asking in different words than the document used. Agricultural questions contain both kinds at once, which is why hybrid search is not optional here.

4.3 Search and retrieval

Vector database	Stores vectors and finds the nearest ones fast. Here: **Qdrant**, with a numpy-on-disk fallback.
Collection / payload	Qdrant's table equivalent, and the metadata stored beside each vector (text, source file, source language).
top-k	How many chunks to hand the model. Set to **8**.
Prefetch	Pull a wider set (30) from each search before fusing, so the fusion has something to work with.
Hybrid search	Running dense and sparse searches and combining the two ranked lists.
RRF Reciprocal Rank Fusion	The combining rule. Each list contributes `1/(k + rank)` per document; scores are summed. Uses *rank*, not raw score, so two incomparable scoring systems can be merged. Consequence: RRF scores are tiny — around 0.01–0.03.
source_lang	Each chunk is tagged with the language it was written in, so retrieval quality can be measured per language.

4.4 Everything else

Language ID	Deciding which language a text is in. Done with **lingua**, a statistical model — script alone cannot separate Hindi from Marathi.
Intent routing	Classifying the question before retrieval: price / weather / spray / scheme / general. A keyword matcher, not a model call — sub-millisecond, free, and debuggable.

Multi-agent	Several model roles collaborating instead of one. Here: three experts plus a chairman.
Consensus / agreement	How much the experts agreed. Partly the chairman's judgement, partly a model-free word-overlap measure.
Rules engine	Plain code making a decision from numbers — the spray window. No model involved, so no invention possible.
STT / TTS	Speech-to-text (Whisper on Groq) and text-to-speech (the browser's own engine — free, offline, and the audio never leaves the phone).
Ablation	Switching a feature off to measure what it was contributing. The central experiment in §10.
IVR	Interactive Voice Response — answering an ordinary phone call. For farmers with no smartphone.

5 The pipeline, stage by stage

Stage 1 — Language detection `farmer/language_detector.py`

Detect once, carry the result as a tag, and reply in it. The detected language is threaded into the generation prompt as `Answer in {language}`.

THE TRAP THIS AVOIDS

Hindi and Marathi are both written in Devanagari. A script check sees the same characters for both and cannot tell them apart — so the system would answer a Marathi farmer in Hindi and look broken. A real language-ID model is a *requirement* here, not a refinement.

Stage 2 — Intent routing `farmer/intent.py`

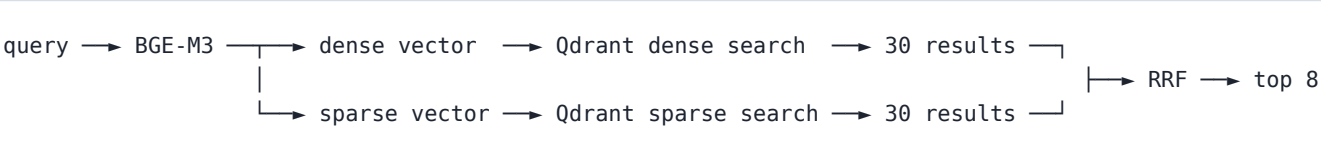
Classify into price / weather / spray / scheme / general **before** retrieving.

WHY THIS EXISTS

Some questions **cannot** be answered from documents. *"What is onion selling for today?"* has no answer in any PDF. Retrieving a storage guide and generating a confident paragraph from it is worse than useless — it looks like an answer. Those questions get routed to live data instead, with a line explaining why.

It is a keyword matcher rather than a model call on purpose: it runs in under a millisecond, costs nothing against a free tier already carrying several calls per question, and when it is wrong you can see exactly which word caused it. Keywords are listed in all three languages — a Marathi speaker asking about price says भाव, not "price".

Stage 3 — Hybrid retrieval `farmer/searcher.py`



One embedding call produces both vectors. Both searches run, RRF fuses the two rankings, the top 8 chunks go to generation.

Stage 4 — Generation `farmer/answer_writer.py`

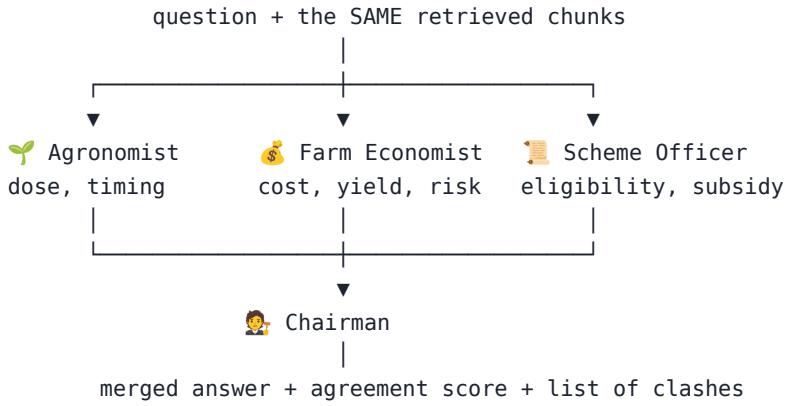
Either one model answering alone, or the council (§6). Everything that talks to Groq goes through a single `chat()` function, so there is exactly one place that knows the API's shape.

BUG FOUND AND FIXED HERE

GPT-OSS spends tokens reasoning before it writes. With the budget too low, `content` comes back as an **empty string — not an error**. Returning that silently produces a blank answer that looks like the model had nothing to say. The code now raises with the cause. Marathi was most at risk, because Devanagari costs more tokens per answer.

6 The council of experts

One model answering alone has nobody to check it. So the same retrieved chunks are read by three specialists with different remits, and a chairman merges them.



Three points to defend in a viva

1. **Different jobs, not different seeds.** Running one prompt three times gives you the same answer three times. Each seat has its own remit, so the opinions genuinely differ — and therefore agreement between them carries information.
2. **Silence is a valid answer.** "How much urea per acre" has nothing in it for the Scheme Officer, so it replies `NO_INPUT` and is dropped from the count. A council where everyone always talks is not a council.
3. **The agreement score is not self-reported.** The chairman gives a number, but word overlap between the opinions is measured separately in `council/agreement.py` with no model involved, and the two are blended 70/30. The UI shows both, so you can see when the chairman is flattering itself.

LIKELY QUESTION

"Isn't this just the same model three times?" — Same weights, different roles and different prompts. The test is whether they ever disagree: they do, and the disagreement is surfaced rather than hidden. If they never disagreed, the consensus score would be meaningless and we would say so.

7 The parts that use no model at all

The most defensible modules in this project are the ones where the language model is deliberately kept out of the decision.

7.1 The spray window `farmer/spray.py`

Every daylight hour of the next two days is scored against plain rules:

- wind low enough that spray does not drift off the field
- no rain forecast within the rainfastness period — rain washes it off
- temperature not so high that it evaporates

Good stretches are named — *"Saturday 07:00–09:00"* — and **every rejected hour says why it was rejected**, so it is not a black box.

WHY COMPUTE IT INSTEAD OF ASKING THE MODEL

Asking a language model which hours are windy is inviting it to invent an answer to a question we can calculate exactly. The hours are worked out in code and handed to the model *already decided*. The model's job is to phrase it, not to determine it.

This is also the economic argument: spraying into wind blows the chemical off the field, and rain within a few hours washes it away. Both mean the money was spent for nothing.

DEMO WARNING

In clear weather the rules reject nothing and the advice becomes "06:00–19:00", which looks like a rubber stamp. Same code, different forecast:

```
Nashik    28/28 hours good  ->  "Friday 06:00-19:00"     clear,   0 mm
Chennai   28/28 hours good  ->  "Friday 06:00-19:00"     clear,   3 mm
Kochi      5/28 hours good  ->  "Saturday 07:00-09:00"   drizzle, 44 mm
```

7.2 Mandi prices `farmer/mandi_prices.py`

Live wholesale prices from data.gov.in, which publishes the AGMARKNET feed — minimum, maximum and modal price per commodity per market, plus arrival quantities.

THE SILENT-FAILURE BUG WORTH TELLING

The API accepts an *old* filter spelling (`filters[state]`) and a *current* one (`filters[State.keyword]`). Guessing wrong does not raise an error — it returns HTTP 200 with zero records. That is exactly how the app ended up quietly showing sample prices while appearing to work.

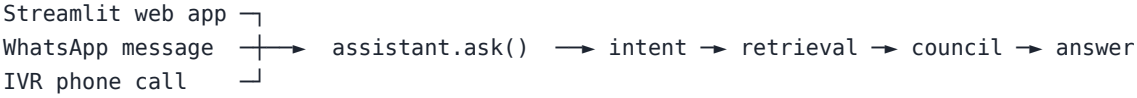
So the client does not guess. It tries each spelling until one returns rows, remembers the winner for the session, and reads every column through a list of aliases. If all filtered calls fail it fetches unfiltered and filters in Python.

Refinement: a timeout is *not* the same failure. Zero-rows means the spelling was wrong, so try the next; a timeout means the host is not answering, and no spelling fixes that. The ladder now stops after two consecutive network failures instead of paying a 20-second timeout six times over.

7.3 Cost calculator

Seeds, fertiliser, labour, irrigation, machinery, transport against yield and selling price → revenue, total cost, profit, cost per kg, break-even price. Arithmetic in Python. No model.

8 Three ways in



`farmer/assistant.py` takes a question and returns an answer **knowing nothing about screens**. Each channel only converts its own message format in and out. That is why adding a channel is a small change rather than a rewrite.

- **WhatsApp** — names its sources, and when agreement is low it *says so in words*, because on a phone there is no panel to expand and an unstated disagreement is invisible.
- **IVR** — records the caller, transcribes with Whisper, reads the answer back with citation markers and bullets stripped out, because those are meaningless out loud.

Stated honestly: the webhook is written against Twilio's documented request and TwiML formats and is fully tested offline, but has never been run against Twilio's live servers.

9 How it is measured

12 questions, each written in all three languages, so cross-language consistency is measured too. Every question is answerable from a real file and names its expected source.

METRIC	WHAT IT MEANS
`hit@k`	A relevant keyword appears anywhere in the top-k chunks. Lenient.
`keyword_recall`	What *fraction* of expected keywords were found. Strict — this is the honest one.
`MRR`	Mean Reciprocal Rank — `1 / rank` of the first relevant chunk. Rewards putting the right thing first.
`same_lang`	Fraction of retrieved chunks in the query's language. A diagnostic, not a target — see §10.
`lang_ok`	Did the answer come back in the requested language (re-detected, not assumed).
`answer_kw`	Did the model actually *use* what retrieval found.
`grounded`	**Fraction of the numbers in the answer that appear in the sources.**

WHY `GROUNDED` IS THE METRIC THAT MATTERS

Advice to a farmer is mostly numbers — 50 kg of urea, ₹6,000, 72 hours. Waffly prose is harmless; a wrong dose is not. So the measure is: of every number the model wrote, how many can be traced back to a source document? Devanagari digits are normalised, so ५० and 50 count as the same number.

This is also what makes the multi-agent claim *testable* rather than decorative.

10 The headline result — the boost that never helped

This is the strongest thing in the project. Lead with it.

What was built

Each chunk is tagged with its source language. A `SAME_LANG_BOOST` was added to nudge evidence in the query's own language slightly higher at fusion time. Reasonable idea.

What was actually happening

```
score = fused_RRF_score      # ~0.01 to 0.03
score = score + 0.10         # several TIMES the entire spread
```

Because RRF scores are tiny, a flat `+0.10` was not a nudge — it was an override. Every same-language chunk outranked every cross-language chunk, regardless of what either one said. A feature described as a tie-breaker was in practice a hard language filter, and it was switching off the cross-lingual retrieval the entire project exists to demonstrate.

How it was caught

`same_lang` came back at **1.00 for every Marathi query** — from a corpus holding only 12 Marathi chunks out of 114. The same handful of chunks was returned whatever the question asked about. The metric exposed it; nothing else would have.

Three conditions measured

CONDITION	EN	HI	MR	MR MRR	MR SAME_LANG
additive `+0.10` (the bug)	0.46	0.33	0.33	0.74	1.00
ratio `×1.10` (fixed)	0.46	0.55	0.48	0.76	0.71
no boost — shipped	**0.46**	**0.60**	**0.54**	**0.81**	**0.64**

Figures are keyword recall unless the column says otherwise.

What it means — three readings

1. **Fixing it helped; removing it helped more.** BGE-M3's shared vector space already did the cross-lingual work. The hand-written heuristic layered on top only subtracted.
2. **English is flat at 0.46 across all three conditions — this is the control.** 83% of the corpus is English, so the boost is nearly a no-op for English queries. A harness that moved English here would be measuring something other than what it claims. *Point at this row first.*
3. **`same_lang` moves against quality** — 1.00 → 0.71 → 0.64 for Marathi *while recall climbs*. A high value means the retriever was *confined*, not confident. It is a diagnostic, not a score to maximise.

STATE THE LIMITATION BEFORE YOU ARE ASKED

n = 12 questions. The additive-versus-anything gap is far too large to be noise. Ratio versus off is suggestive, not settled. Saying this yourself is worth more than being caught on it.

HOW TO TELL THIS STORY

"We added multi-agent consensus" is a claim anyone can make. **"Our own heuristic was silently disabling the cross-lingual retrieval the project is built on; the metrics caught it, we measured three conditions, and we deleted the feature"** is a result — and it required building the measurement *before* trusting the design. The negative result is the stronger half.

11 Design decisions — why this and not that

DECISION	REASON
BGE-M3, not separate per-language models	One shared vector space means a Marathi query matches an English document directly. Separate models would need a translation hop, which is the thing being avoided.
No translate-to-English step	Every hop loses domain vocabulary, adds latency and adds cost.
lingua for language ID	Pure Python, no compiler, no model download — and it separates Hindi from Marathi, which script cannot.
Keyword intent router, not a model	Sub-millisecond, free, and inspectable when wrong.
Rules for the spray window	A computable question should be computed. Explainable, and cannot hallucinate.
Qdrant with a numpy fallback	A free Qdrant cluster is switched off after a few idle weeks. Finding that out on the morning of a demo is a bad time to find it out.
Groq free tier	Zero cost. The constraint it imposes is real and is designed around, not ignored.
Browser TTS, not an API	No key, no cost, works offline once loaded, and the audio never leaves the phone.
Streamlit	The whole UI is Python. The project's value is in retrieval and evaluation, not in a frontend build.
Index committed to the repo	A free Space sleeps and wakes with a clean disk. An index built at runtime is gone by the next visitor.

12 Failure handling — it degrades, it does not crash

WHEN THIS FAILS	THIS HAPPENS INSTEAD
One council expert errors	Marked failed, dropped; the rest continue
Only one expert speaks	Chairman is skipped entirely
The whole council falls over	Plain single-model answer is served
Qdrant cluster is gone	Numpy index on disk, same RRF fusion
Price API unreachable	Clearly labelled sample data + "NOT today's rates"
Soil lookup times out	Daily brief drops the soil card, keeps the weather
Index is empty	Evaluation refuses to run and names the fix
Model returns empty content	Raises with the cause instead of a blank answer

WHY THIS IS WORTH A SLIDE OF ITS OWN

Every one of those rows is a real failure that actually happened during development. If something breaks during the demo, **name the fallback that fired** — that is the system working as designed, and saying so is stronger than reloading and hoping.

13 Known limits — say these first

- **The corpus is illustrative.** The 24 documents are written for the demo and several say so in their own first line. Fine for a demo; **not** fine for a farmer acting on a dose. Real ICAR/KVK material must replace them before any real use.
- **Keyword recall tops out around 0.46–0.60.** Retrieval finds something relevant almost every time (`hit@k` = 1.00) but only about half the expected facts. Chunk size and `top-k` are where to look.
- **No crop calendar.** The profile stores crops but not sowing dates, so advice is not staged by growth stage.
- **The Twilio webhook has never run live.**
- **A public deployment spends one shared API key** on every visitor. No auth, no per-user limiting.
- **n = 12** on the evaluation set.

14 Viva questions and answers

Q — WHAT IS RAG AND WHY USE IT HERE?

Retrieve relevant passages from a trusted corpus, then make the model answer from them rather than from memory. Used here because agricultural advice is mostly numbers, a hallucinated dose costs a season, and every answer needs to name its source.

Q — HOW DOES A MARATHI QUESTION FIND AN ENGLISH DOCUMENT?

BGE-M3 is multilingual and places text from all three languages in one shared vector space, so युरिया and "urea" land near each other. No translation step is involved.

Q — WHY NOT DETECT LANGUAGE FROM THE SCRIPT?

Hindi and Marathi both use Devanagari. The characters are identical, so a script check cannot distinguish them and would answer in the wrong language. lingua is a statistical model and can.

Q — WHY BOTH DENSE AND SPARSE VECTORS?

Dense captures meaning and handles paraphrase; sparse captures exact strings like scheme names and chemical names. Farming questions contain both. BGE-M3 emits both from one pass, so there is no separate BM25 index to maintain.

Q — WHAT IS RRF AND WHY USE IT?

Reciprocal Rank Fusion. Each result list contributes `1/(k+rank)` per document and the scores are summed. It uses rank rather than raw score, so two scoring systems that are not comparable can still be merged. Its scores are small — about 0.01–0.03 — which is exactly what made our boost bug possible.

Q — WHAT WAS YOUR MOST INTERESTING FINDING?

That our own same-language boost was a hard filter rather than a tie-breaker, and that removing it entirely beat both fixing it and keeping it. The encoder did not need our help. The English control row proves the harness was measuring the right thing.

Q — IS THE COUNCIL REAL, OR ONE MODEL THREE TIMES?

Same weights, three distinct remits and prompts. They disagree, the disagreement is surfaced, and the agreement score is partly computed without any model so the chairman cannot inflate it.

Q — HOW DO YOU KNOW THE MODEL IS NOT MAKING NUMBERS UP?

The `grounded` metric: the fraction of numbers in the answer that appear in the retrieved sources, with Devanagari digits normalised.

Q — WHAT HAPPENS WHEN A SERVICE GOES DOWN?

Every path degrades rather than crashing — see §12. During development Qdrant was deleted, SoilGrids went down, data.gov.in became unreachable and Groq retired our model. The app survived all four.

Q — WHAT WOULD YOU DO NEXT?

Real ICAR/KVK documents instead of illustrative ones; crop-calendar and growth-stage filtering; a larger evaluation set than 12 questions; and a disease-identification path from a photo.

15 One-page cheat sheet

THING	VALUE
Languages	Marathi, Hindi, English — in, out, and the whole interface
Embedding model	BGE-M3, 1024 dimensions, dense + sparse
Vector store	Qdrant, with numpy-on-disk fallback (same RRF)
Fusion	Reciprocal Rank Fusion, prefetch 30 each, top-k 8
Generation	GPT-OSS 120B on Groq, temperature 0.2, reasoning_effort low
Speech	Whisper (in) · browser TTS (out)
Corpus	24 documents → 114 chunks · 20 en, 2 hi, 2 mr
Eval set	12 questions × 3 languages = 36 rows
Headline result	Same-language boost removed; hi recall 0.33 → 0.60, mr 0.33 → 0.54
Channels	Streamlit · WhatsApp · IVR, all over one `assistant.ask()`
Free-tier ceiling	200,000 tokens/day — a design constraint, not a footnote
Total cost	₹0

IF YOU REMEMBER THREE SENTENCES

One. A multilingual encoder lets a Marathi question retrieve an English document directly, so nothing is translated and nothing is lost. **Two.** Anything computable is computed — spray hours, costs, prices — and the model only phrases the result. **Three.** We measured our own feature, found it was doing the opposite of what we intended, and removed it.